



Práctica Extra 2

Organización y Arquitectura de Computadoras

Docente: M.C. Garcia Rocha Jose Isabel

Alumno:

- Chaparro Herrera Hugo Giovanni - 2200357

Tijuana, Baja California a 30 de noviembre de 2025

Objetivo

El alumno se familiarizará con el intérprete 80X86 (modo 16bits) sobre la plataforma T-Juino. Esto con el fin de desarrollar programas en lenguaje ensamblador para dicha plataforma.

Introducción

La tarjeta T-Juino es una plataforma de hardware basada en un PCB con un microcontrolador y un entorno de desarrollo, diseñada para facilitar el uso de microcontroladores en proyectos multidisciplinarios.

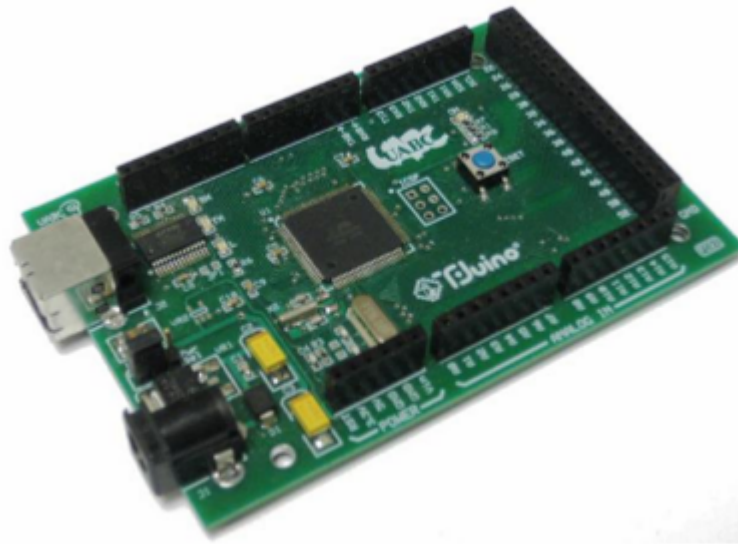


Figura 1. Tarjeta T-Juino.

T-Juino fue diseñado para ser compatible con la plataforma libre llamada Arduino (específicamente Arduino Mega) y el sistema SM8088 de UABC. El hardware consiste de una placa con un microcontrolador Atmel AVR con sus puertos de entrada/salida expuesto mediante conectores. El microcontrolador utilizado en el diseño es el Atmega1280 y para el desarrollo de software se puede utilizar entornos de desarrollo libres como Arduino IDE, Wiring o WinAVR+ Programmer notepad.

Es importante remarcar que una diferencia con la plataforma Arduino es que en T-Juino se cuenta con una máquina virtual (VM88) del procesador 80x86 de 16 bits desarrollada en UABC. Esta máquina lo hace compatible con los programas ejecutables que fueron desarrollados para la plataforma SM8088 basada en el procesador clásico de Intel 80C88.



Desarrollo

Para instalar el intérprete realice los siguientes pasos:

1. Ejecutar la aplicación Xloader, colocar el puerto, seleccionar la tarjeta y cargar el intérprete.

Para utilizar MTTTY.

1. Configurar Baud a 19200, paridad none, data bits 8 y stop bit 1.
2. Dar click en conectar.
3. Presionar cualquier tecla al aparecer el símbolo ?.
4. Presionar I y enter para cargar el archivo .com o .bin a cargar.
5. Presionar g para iniciar el programa.
6. Para terminar el programa presionar el push-button de la tarjeta.

Comandos del modo monitor:

Los comandos aceptados en este modo se pueden mostrar mediante la tecla ? que corresponde al comando de help.

Nota: no todos los comandos desplegados están implementados y algunos se encuentran en proceso de su implementación.

Los comandos del modo monitor se muestran a continuación.

*clear	C <range>
dump	D <range>
*enter	E address <list>
*fill	F range list
go	G
help	?
input	I port
load	L
*move	M range address
output	O port value
ports	P
quit	Q
register	R <register value>
trace	T
unassemble	U <range>
version	V
zero (rst)	Z
* - in process	



Comandos

- d** – *dump*: hace un desplegado del contenido de un rango de memoria, el desplegado se hace en formato decimal y ASCII (si el contenido es desplegable).
- g** – *go*: ejecuta el programa que se encuentra cargado en la memoria a partir de la dirección 100h.
- ?** – *help*: muestra la lista de comandos permitidos en el modo monitor.
- i** – *input*: leer un byte de un puerto dado.
- l** – *load*: carga en memoria un archivo (programa) a partir de la dirección 100h.
- o** – *output*: escribe un byte a un puerto dado.
- p** – *ports*: muestra el valor actual de los puertos de entrada-salida (E/S).
- q** – *quit*: sale del modo monitor e inicia el modo de ejecución.
- r** – *registers*: muestra el valor actual de todos los registros del procesador, además puede ser usado para modificar los registro del procesar.
- t** – *trace*: ejecutar la instrucción apuntado por el CS:IP y muestra los valores resultantes de los registros.
- u** – *unassemble*: desensambla un rango de memoria dado mostrando los mnemónicos correspondiente.
- v** – *version*: muestra la versión del interprete que se encuentra cargada en T-Juino.
- z** – *reset*: realiza un reset sobre el intérprete inicializando todos sus registros en cero excepto IP que lo inicializa en 100h.

- 1) Basándose en el listado 1 crear un archivo texto llamado p10.asm.

Listado 1

```
.model tiny

;----- Insert INCLUDE "filename" directives here
;----- Insert EQU and = equates here

locals

.data
    Mens DB 'Hola Mundo',10,13,0

.code
    org 100h

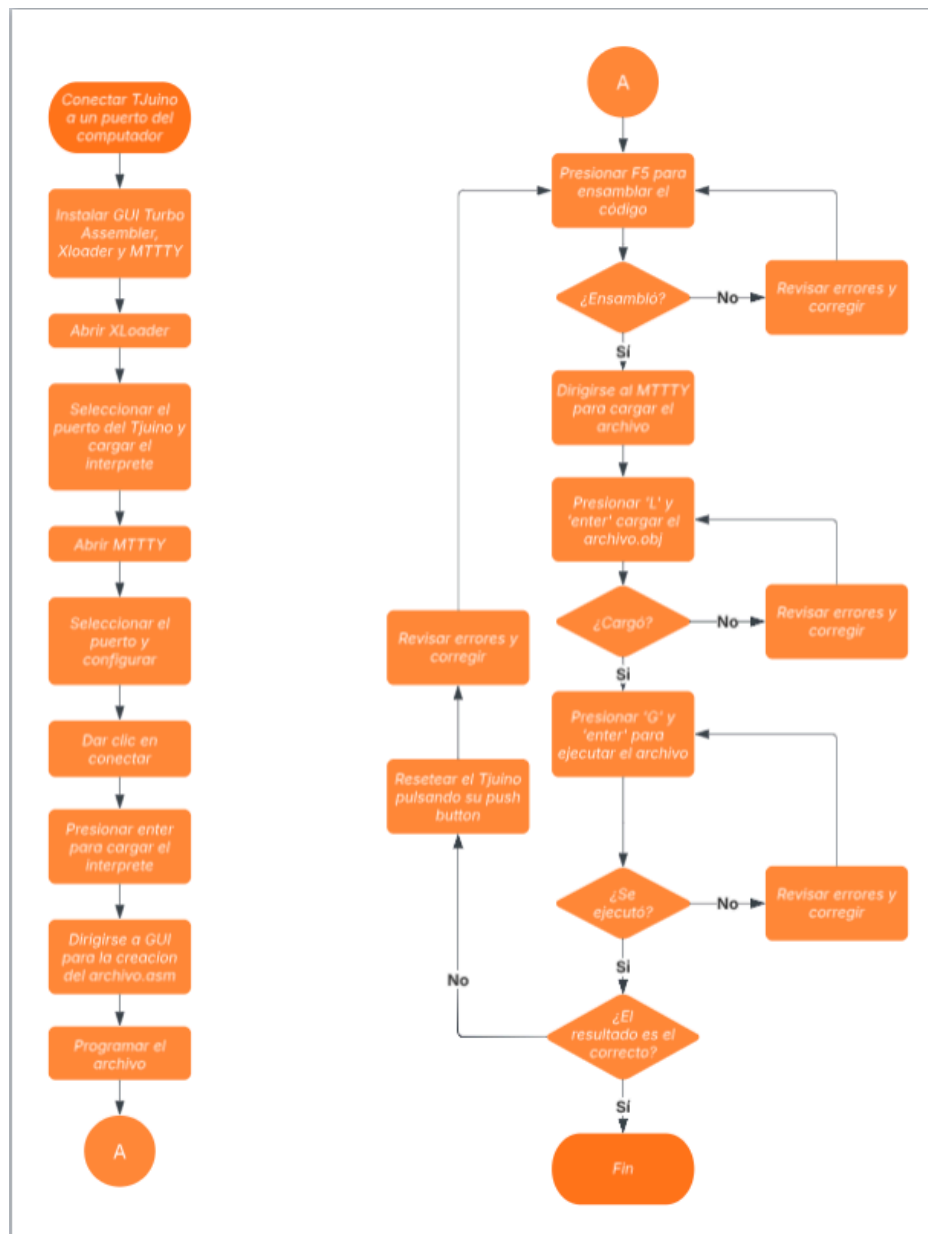
; *****
;   Procedimiento principal
; *****
principal proc
    mov     sp,0fffh           ; inicializar SP (Stack Pointer)
    @@ini0: mov     dx,1
    @@ini1: mov     cx,dx
    @@sigue: mov     al,'x'
             call    putchar
             loop    @@sigue
             mov     al,10
             call    putchar
             mov     al,13
             call    putchar
             inc     dx
             cmp     dx,20
             jbe     @@ini1
             jmp     @@ini0
             ret
    endp

; *****
;   procedimientos
; *****
putchar proc
    push    ax
    push    dx
    mov     dl,al
    mov     ah,2               ; imprimir caracter DL
    int     21h               ; usando servicio 2 (ah=2)
    pop     dx                 ; del int 21h
    pop     ax
    ret
    endp

end principal                ; fin de programa (file)
```



1. Instalar GUI Turbo Assembler v3.0.1 versión 64 bits si es necesario.
 2. Ejecutar el programa, abrir la sección de preferencias y modificar los parámetros del linker, eliminar los que están ahí presentes y agregar '/t'.
 3. Cargar el archivo .asm
 4. Presionar F5 para ensamblar y enlazar el archivo, se genera el archivo .obj y posteriormente el archivo .com.
 5. Cargar este último archivo con 'l' en la placa y ejecutarlo con 'g'.
- 2) Elaborar un diagrama de flujo detallando el proceso para realizar todas las actividades anteriormente presentadas, paso a paso.





3) Implementar **printHex** y **printDecimal** en ensamblador 8086 y correrlo en la tarjeta.

Nota: **printHex** y **printDecimal** sólo deben imprimir las cadenas resultantes, no deben almacenarlas.

printHex:

```
printHex  PROC
    push ax
    push bx
    push cx
    push dx
    mov dx, ax
    mov bx, 0Fh
    mov cl, 12
    @@nxt: shr ax, cl
    @@msk: and ax, bx                ; Mascara, solo tener nibble mas bajo
    cmp al, 9                      ; Comparar si es numero o letra
    jbe @@menor                    ; Si AL <= 0xAh
    add al, 7                      ; Mascara por si es letra
    @@menor: add al, '0'            ; Convertir en caracter ASCII
    call putchar

    cmp cl, 0
    je @@fin

    mov ax, dx                    ; Reestablecer valor en AX
    sub cl, 4                    ; Se le resta 4 para avanzar en nibble
    cmp cl, 0                    ; Ver si aun quedan corrimientos
    ja @@nxt                      ; Si aun quedan
    je @@msk

    @@fin: pop DX
    pop CX
    pop BX
    pop AX
    ret
ENDP
```

Se manda a llamar colocando en AX el numero que queremos imprimir en HEX, en este caso, le mandamos un 15d para que nos muestre en pantalla un Fh.

9	<code>mov ax, 15d</code>	<code>; AX = numero a imprimir</code>
10	<code>call printHex</code>	
11	<code>call salto</code>	
12		

```
-----
>g
000F
```



printDec:

```
printDecimal    PROC
    push ax
    push bx
    push cx
    push dx
    mov bx, 0      ; Contador de Longitud
    mov cx, 10     ; Divisor Base 10

    @@convertir:
        cmp ax, 0      ; se compara AX para ver si la base es 0
        je @@imprimir

        mov dx, 0      ; resetea DX para almacenar los residuos
        div cx         ; se divide entre la base 10

        add DL, '0'    ; se convierte a ASCII
        push dx        ; se guarda el caracter en la pila
        inc bx         ; se incrementa en 1 el cont de len
        jmp @@convertir

    @@imprimir:
        cmp bx, 0      ; se compara para ver si ya terminamos la impresion
        je @@terminar

        dec bx         ; len - 1
        pop dx         ; extraemos el dato de la pila
        mov al, dl      ; movemos el dato DL a AL para imprimirlo con putchar
        call putchar
        jmp @@imprimir

    @@terminar:
        pop dx
        pop cx
        pop bx
        pop ax
        ret
ENDP
```

De igual forma, en AX le mandamos ahora un 0Fh, para que en pantalla nos muestre el 15.

```
mov ax, 0Fh      ; AX = Numero a imprimir
call printDecimal
call salto
```

15
0?>



Código en ASM

```
.model tiny
locals
.code
    org 100h

main    PROC
    mov sp, 0FFFh

    mov ax, 15d    ; AX = numero a imprimir
    call printHex
    call salto

    mov ax, 0Fh    ; AX = Numero a imprimir
    call printDec
    call salto

    ; sys_exit
    mov ah, 04Ch
    int 21h
    ret
    endp

;-----
printHex    PROC
    push ax
    push bx
    push cx
    push dx
    mov dx, ax
    mov bx, 0Fh
    mov cl, 12

    @@nxt: shr ax, cl
    @@msk: and ax, bx    ; Mascara, solo tener nibble mas bajo
               cmp al, 9    ; Comparar si es numero o letra
               jbe @@menor    ; Si AL <= 0xAh
               add al, 7    ; Mascara por si es letra
    @@menor: add al, '0'    ; Convertir en caracter ASCII
               call putchar

               cmp cl, 0
               je @@fin

               mov ax, dx    ; Reestablecer valor en AX
               sub cl, 4    ; Se le resta 4 para avanzar en nibble
               cmp cl, 0    ; Ver si aun quedan corrimientos
               ja @@nxt    ; Si aun quedan
               je @@msk

    @@fin: pop dx
endproc
```




```
        pop CX
        pop BX
        pop AX
        ret

    ENDP
;-----
printDec  PROC
    push ax
    push bx
    push cx
    push dx
    mov bx, 0      ; Contador de Longitud
    mov cx, 10     ; Divisor Base 10

    @@convertir:
        cmp ax, 0      ; se compara AX para ver si la base es 0
        je @@imprimir

        mov dx, 0      ; resetea DX para almacenar los residuos
        div cx         ; se divide entre la base 10

        add DL, '0'    ; se convierte a ASCII
        push dx        ; se guarda el caracter en la pila
        inc bx         ; se incrementa en 1 el cont de len
        jmp @@convertir

    @@imprimir:
        cmp bx, 0      ; se compara para ver si ya terminamos la impresion
        je @@terminar

        dec bx        ; len - 1
        pop dx         ; extraemos el dato de la pila
        mov al, dl     ; movemos el dato DL a AL para imprimirlo con putchar
        call putchar
        jmp @@imprimir

    @@terminar:
        pop dx
        pop cx
        pop bx
        pop ax
        ret

    ENDP
;-----
putchar  PROC
    push ax
    push dx
    mov dl, al
    mov AH, 02h
    int 21h
```



```
    pop dx
    pop ax
    ret
    ENDP
;-----
salto    PROC
    push ax
    push dx
        mov dl, 13
        mov ah, 02h
        int 21h

        mov dl, 10
        mov ah, 02h
        int 21h
    pop dx
    pop ax
    ret
    ENDP
end main
```



Dificultades durante el desarrollo

Durante el desarrollo de la práctica, las mayores dificultades presentadas fueron como se comportaba la placa al querer programar en ella, ya que, era bastante común que surgieran problemas al realizar código. Estos problemas principalmente eran de cómo funcionaba el Tjuino, ya que, a veces si se ensamblaba y otras no cuando directamente no se le había movido ni un comentario al código.

Referencias Bibliográficas

- [02. NASMx86: partes del código fuente](#)
- [Guide to x86 Assembly](#)
- [Using as - Assembler Directives](#)
- [NASM Assembly Language Tutorials - asmtutor.com](#)
- [Assembly - System Calls](#)